

DATA SCIENCE PROJECT REPORT

Titanic Survival Analysis

Exploratory Data Analysis and Logistic Regression Classification

EDWARD APERSIL | 01259814B

Dataset: Kaggle Titanic training dataset | Prepared from main.ipynb

Table of Contents

1. Cover Page
2. Table of Contents
3. Introduction
4. Dataset Description
5. Methodology
6. Results
7. Discussion
8. Conclusion
9. References
10. Appendix: Python Code

1. Introduction

This report presents a data science analysis of the Titanic passenger dataset. The main objective is to understand the passenger characteristics associated with survival and to build a simple classification model that predicts whether a passenger survived. The analysis combines data cleaning, exploratory data analysis, correlation analysis, and logistic regression modelling.

The Titanic dataset is useful for introductory data science because it contains a mixture of numeric, categorical, complete, and missing values. It also has a clear binary target variable, Survived, which makes it appropriate for a classification task.

2. Dataset Description

The original training dataset contains 891 passenger records and 12 columns. After preprocessing, Cabin was removed because it had a high missing-value rate, Age was filled using the median age, and Embarked was filled using its most frequent value, S.

Feature	Description	Type
PassengerId	Unique passenger identifier	Numeric identifier
Survived	Target variable: 0 = did not survive, 1 = survived	Binary
Pclass	Ticket class: 1st, 2nd, or 3rd	Ordinal numeric
Name	Passenger name	Text
Sex	Passenger sex	Categorical
Age	Passenger age in years	Numeric
SibSp	Number of siblings/spouses aboard	Numeric
Parch	Number of parents/children aboard	Numeric
Ticket	Ticket number	Text
Fare	Passenger fare	Numeric
Cabin	Cabin number, later dropped due to missingness	Text
Embarked	Port of embarkation: C, Q, or S	Categorical

Column	Missing Count	Missing Percentage
Age	177	19.9%
Cabin	687	77.1%
Embarked	2	0.2%

3. Methodology

3.1 Data Cleaning

The analysis began by checking dataset dimensions, column names, data types, missing values, and duplicate rows. Age was imputed with the median because it is less sensitive to extreme values than the mean. Cabin was removed because most records did not contain a cabin value. Embarked was imputed using the mode.

3.2 Exploratory Data Analysis

Exploratory analysis examined the distribution of age, passenger class, fare, sex, embarkation point, and survival. The notebook also visualized age by class, age versus fare, a correlation heatmap, and pairwise relationships among selected numeric variables.

3.3 Machine Learning

A logistic regression classifier was used because the target variable is binary. The selected predictor variables were Pclass, Sex, Age, SibSp, Parch, and Fare. Sex was encoded numerically, with male mapped to 0 and female mapped to 1. The data was split into 80% training data and 20% testing data using a fixed random state for reproducibility.

4. Results

4.1 Descriptive Statistics

Variable	Mean	Median	Minimum	Maximum
Age	29.36	28.00	0.42	80.00
Fare	32.20	14.45	0.00	512.33
SibSp	0.52	0.00	0.00	8.00
Parch	0.38	0.00	0.00	6.00

Category	Count	Percentage
Survived	342	38.4%
Did not survive	549	61.6%
Male	577	64.8%
Female	314	35.2%
Third class	491	55.1%
Second class	184	20.7%
First class	216	24.2%

4.2 Survival Patterns

Group	Survival Rate
First class passengers	63.0%
Second class passengers	47.3%
Third class passengers	24.2%
Female passengers	74.2%
Male passengers	18.9%

The results show a strong survival gap by sex and passenger class. Female passengers had a much higher survival rate than male passengers, and first-class passengers had a substantially higher survival rate than third-class passengers.

4.3 Correlation and Model Performance

Relationship	Correlation
Pclass and Fare	-0.549
SibSp and Parch	0.415
Pclass and Survived	-0.338
Fare and Survived	0.257
Age and Survived	-0.065

Metric	Value
Training set size	712 records
Testing set size	179 records
Accuracy	0.8101
Confusion matrix	[[92, 13], [21, 53]]
Precision for non-survival class	0.81
Recall for non-survival class	0.88
Precision for survival class	0.80
Recall for survival class	0.72

5. Discussion

The analysis supports the idea that survival was not randomly distributed. Passenger class and fare both point toward socioeconomic differences in survival outcomes, while sex shows the most visible survival gap. The class gradient is especially clear: first-class passengers survived at about 63%, compared with about 24% for third-class passengers.

The logistic regression model performed well for a simple baseline, reaching 81.01% accuracy on the test set. However, the model recalled non-survivors better than survivors, meaning it was more successful at identifying passengers who did not survive. This is common when the dataset has an imbalanced target, because there are more non-survivors than survivors.

Several improvements are possible. Feature engineering could extract title from Name, family size from SibSp and Parch, and deck information from Cabin where available. Categorical encoding for Embarked could also be included. Finally, models such as random forests, gradient boosting, or regularized logistic regression with scaling could be compared against the current baseline.

6. Conclusion

This project demonstrated the full data science workflow on the Titanic dataset: data acquisition, cleaning, exploratory analysis, statistical interpretation, and machine learning classification. The findings show that sex, passenger class, and fare were important indicators of survival. The logistic regression model provided a clear and interpretable baseline with strong test performance for an introductory classification task.

7. References

- Kaggle. Titanic - Machine Learning from Disaster dataset.
- McKinney, W. pandas: a foundational Python library for data analysis.
- Harris, C. R., et al. Array programming with NumPy.
- Pedregosa, F., et al. Scikit-learn: Machine Learning in Python.
- Hunter, J. D. Matplotlib: A 2D graphics environment.
- Waskom, M. seaborn: statistical data visualization.

8. Appendix: Python Code

The appendix below contains the project code structure from the notebook, with the feature matrix consistently named X in the modelling section.

```
# -----#import libraries for
data acquisition
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

#loading the data set

df = pd.read_csv('train.csv')

#Dataset Dimension
print("Dataset dimensions (rows, columns", df.shape)

#print Column name
print("\nColumn names:")
print(df.columns.tolist())

df.head()

print(df.dtypes)

# Count missing values per column
missing = df.isnull().sum()
missing_percent = (missing / len(df)) * 100

missing_table = pd.DataFrame({
    'Missing Percentage': missing_percent,
    'Missing Count': missing
})

print(missing_table)

df['Age'] = df['Age'].fillna(df['Age'].median())
df = df.drop(columns=['Cabin'], errors='ignore')
df['Embarked'] = df['Embarked'].fillna(df['Embarked'].mode()[0])

#prints duplicates
print("Duplicated rows:", df.duplicated().sum())
print(df.isnull().sum())
print("\nNew dimensions:", df.shape)

sns.set_style('whitegrid')

#Histogram of passenger ages
plt.figure(figsize=(8, 5))
plt.hist(df['Age'], bins=20, color='steelblue', edgecolor='black')
plt.title("Age Distribution of Passengers")
plt.xlabel("Age (in years)")
plt.ylabel("Number of Passengers")
plt.show()

plt.figure(figsize=(8, 5))
class_counts = df['Pclass'].value_counts().sort_index()
plt.bar(class_counts.index.astype(str), class_counts.values,
        color=['goldenrod', 'silver', 'saddlebrown'])
plt.title('Passenger Class Distribution')
plt.xlabel('Passenger Class')
plt.ylabel('Number of Passengers')
plt.show()

plt.figure(figsize=(8, 5))
```

```

sns.boxplot(x='Pclass', y='Age', data=df, hue='Pclass', palette='Set2', legend=False)
plt.title('Age Distribution by Passenger Class')
plt.xlabel('Passenger Class')
plt.ylabel('Age (years)')
plt.show()

plt.figure(figsize=(8, 5))
plt.scatter(df['Age'], df['Fare'], alpha=0.5, color='teal')
plt.title('Age versus Fare')
plt.xlabel('Age (years)')
plt.ylabel('Fare (GHC)')
plt.show()

plt.figure(figsize=(8, 6))
numeric_df = df.select_dtypes(include=[np.number]).drop(columns=['PassengerId'])
sns.heatmap(numeric_df.corr(), annot=True, cmap='coolwarm', fmt='.2f',
            vmin=-1, vmax=1)
plt.title('Correlation Heatmap of Numerical Variables')
plt.show()

selected = df[['Survived', 'Pclass', 'Age', 'Fare']]
sns.pairplot(selected, hue='Survived', palette={0: 'crimson', 1: 'seagreen'})
plt.suptitle('Pairplot of Selected Numerical Variables', y=1.02)
plt.show()

df.describe()

df.describe(include='str')

for col in ['Survived', 'Pclass', 'Sex', 'Embarked']:
    print(f"\n--- {col} ---")
    print(df[col].value_counts())
    print(df[col].value_counts(normalize=True).round(3) * 100)

numeric_df = df.select_dtypes(include=[np.number]).drop(columns=['PassengerId'])
corr_matrix = numeric_df.corr()
print(corr_matrix.round(3))

# Unstack the matrix into pairs, remove self-correlations (which are always 1.0)
corr_pairs = corr_matrix.unstack()
corr_pairs = corr_pairs[corr_pairs < 1.0]

print("Strongest positive:", corr_pairs.idxmax(), round(corr_pairs.max(), 3))
print("Strongest negative:", corr_pairs.idxmin(), round(corr_pairs.min(), 3))

print(df.groupby('Pclass')['Survived'].mean().round(3) * 100)

print(df.groupby('Sex')['Survived'].mean().round(3) * 100)

model_df = df.copy()

model_df['Sex'] = model_df['Sex'].map({'male': 0, 'female': 1})

features = ['Pclass', 'Sex', 'Age', 'SibSp', 'Parch', 'Fare']
X = model_df[features]
y = model_df['Survived']

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("Training set:", X_train.shape)
print("Testing set:", X_test.shape)

from sklearn.linear_model import LogisticRegression

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

```



```
y_pred = model.predict(X_test)

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

print("Accuracy:", round(accuracy_score(y_test, y_pred), 4))

cm = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:\n", cm)

print("\nClassification Report:\n", classification_report(y_test, y_pred))
```